

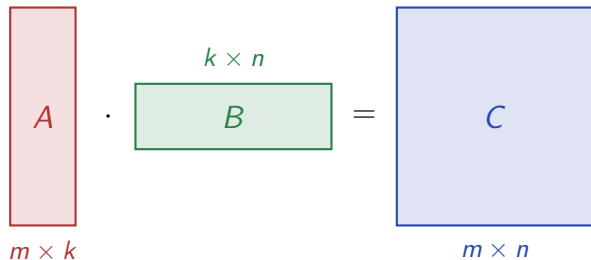
Cache-Optimal Tiling for Mixed-Precision Matrix Multiplication

Areg Mkhitarian Regev Ginzburg

Technion – Israel Institute of Technology

Supervisor: Alon Amid

Matrix Multiply is Memory-Bound



Problem: naive loop reads A and B from RAM on every pass.

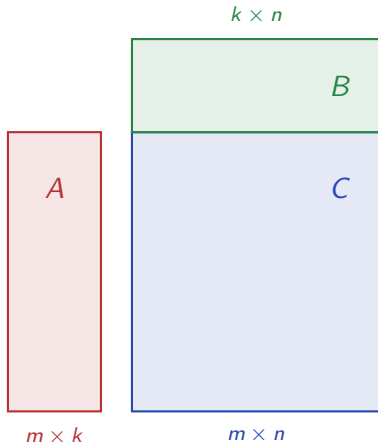
256×256 fp64: ≈ 0.5 MB

Typical L1 cache: 16–64 KB

$\Rightarrow 8\times\text{--}32\times$ **larger than L1**

Fix: *tile* the loop — keep one C tile resident in L1.

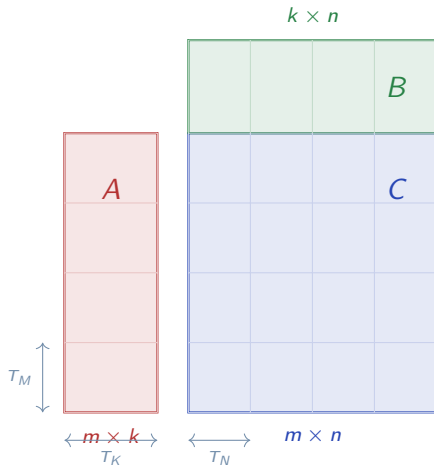
C-Stationary Tiling: Step by Step



$$C = A \cdot B$$

For large m, n, k — none fit in L1.

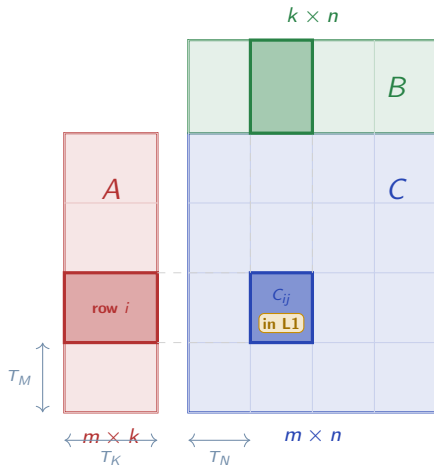
C-Stationary Tiling: Step by Step



Tile shape: $T_M \times T_N \times T_K$

Each matrix divided into a grid of tiles.

C-Stationary Tiling: Step by Step

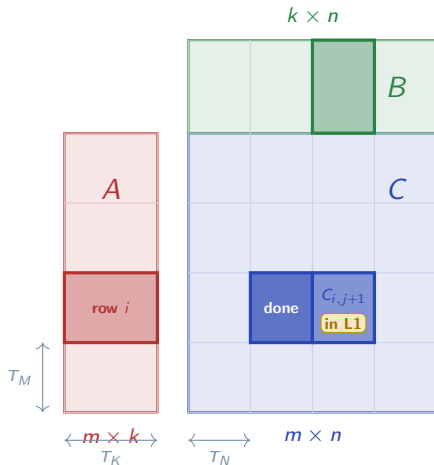


Fix C_{ij} in L1.

Stream A row-band and B col-band through it.

$$C_{ij} += A_{\text{row } i} \times B_{\text{col } j}$$

C-Stationary Tiling: Step by Step



C_{ij} done — move to B col $j+1$.

A row-band stays in L1.

No reload needed.

Standard Assumption: Square Tiles

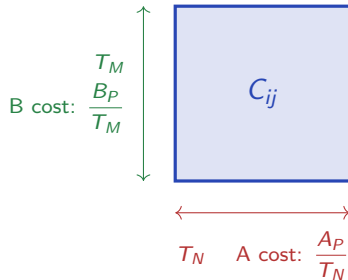
Each A element is reused T_N times (once per C column).

Each B element is reused T_M times (once per C row).

$$T = MNK \left(\frac{A_P}{T_N} + \frac{B_P}{T_M} \right)$$

For equal precision ($A_P = B_P$), minimize $\frac{1}{T_M} + \frac{1}{T_N}$ subject to $T_M T_N \leq S$:

$$T_M^* = T_N^* \quad (\text{square tiles})$$



For $A_P=B_P$: both costs equal at $T_M=T_N$.

Asymmetric Precision $\Rightarrow$ Asymmetric Tiles

Let $\rho = B_P/A_P$ (bytes per element).

When $B_P < A_P$: B is cheaper to load $\Rightarrow$ use wider tiles.

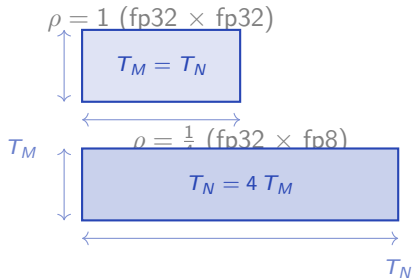
$$T = MNK \left(\frac{A_P}{T_N} + \frac{B_P}{T_M} \right)$$

Balance A and B traffic:

$$\frac{A_P}{T_N} = \frac{B_P}{T_M} \Rightarrow \boxed{\frac{T_N^*}{T_M^*} = \frac{A_P}{B_P} = \frac{1}{\rho}}$$

Example: fp32 $\times$ fp4 ($\rho = \frac{1}{8}$):

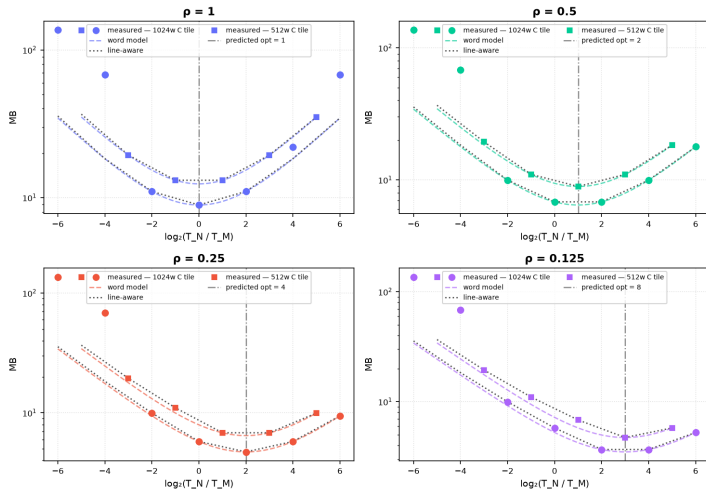
$$T_N^* = 8 T_M^*$$



Same cache budget S — wider tile, better B reuse.

Traffic Minimum at the Predicted Tile

L1 reads (BytesIn) vs paper prediction — fully-associative cache
 $m=n=k=256$, $T_K=256$, outer-products order



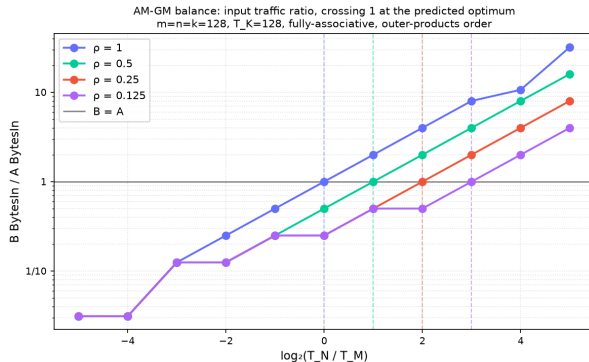
Each curve shows L1 reads as a function of tile aspect ratio T_N/T_M .

Dashed line: paper's prediction $T_N/T_M = 1/\rho$.

The measured minimum lands **exactly** at the predicted optimum for all four ρ values.

Simulation matches formula within $\sim 2\%$.

B/A Balance: Exact Confirmation



Each curve is the ratio of B reads to A reads as T_N / T_M varies.

At the paper's predicted tile, every curve crosses $B/A = 1$ **exactly**.

Traffic saved vs. square tile:

ρ	saving	
1	0%	(symmetric)
0.5	0%	
0.25	18%	
0.125	36%	

Traffic $\neq$ Time

The paper minimizes *bytes transferred*:

$$T_{\text{traffic}} = MNK \left(\frac{A_P}{T_N} + \frac{B_P}{T_M} \right)$$

But cycles $\neq$ bytes.

Cache latency, bandwidth limits, and prefetch behavior all break the proportionality between traffic and runtime.

We need a *cycle-accurate* model.

Enter the PRNG FIFO

B elements are generated *on-chip* by a hardware random number generator, streamed directly into the compute unit.

B never touches the cache.

B has no memory address.

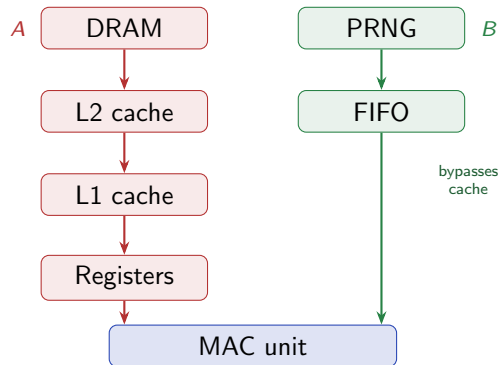
The B_P/T_M traffic term **vanishes**.

Instead, B introduces a new cost: *on-chip generation time*.

generation cost = g_c cycles / element

Our model must account for g_c , not B_P .

The PRNG FIFO: B Without Memory



Random matmul: $C = A \cdot B$ where B is a pseudo-random matrix generated on-chip — too large to store in memory.

Use cases: random projections, privacy-preserving ML, approximate algorithms.

g_c : cycles to produce one B element. The only B -related cost — no cache misses, no bandwidth.

Two independent bottlenecks

A-bound: load A from memory

B-bound: wait for PRNG (g_c cy/elem)

Runtime = whichever is slower.

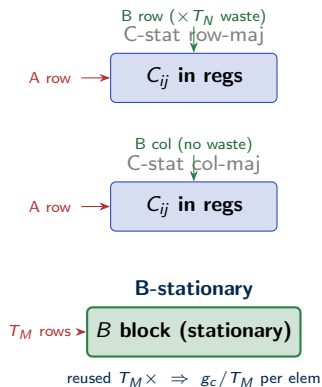
Which Loop Order Fits the FIFO?

B elements emerge from the PRNG in a **fixed order**
— you cannot skip or replay elements.

Three loop structures are possible:

Mode	In regs	B cost / elem
C-stat row-maj	C_{ij} tile	$g_c \times T_N$ (waste)
C-stat col-maj	C_{ij} tile	g_c
B-stationary	B block (outer)	g_c / T_M

Only B-stationary amortizes generation cost
across the T_M output rows.



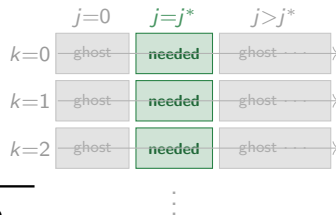
C-Stationary Row-Major: Ghost Read Problem

C tile fixed in registers; inner K -loop streams
A rows and B sub-columns through it.

Problem: the FIFO generates B in
row-major order — all N/T_N column groups
of row k before row $k+1$.

C tile at column j needs *one* group per k
step.

The other $N/T_N - 1$ groups are generated
and discarded — **ghost reads**.



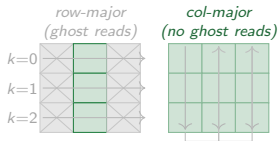
	$g_c=0$	$g_c=10$	$g_c=100$	N/T_N groups generated per k step; only 1 is useful
B-stat (ref)	ref	ref	ref	
C-stat row-maj	$2\times$ faster	$1.25\times$ faster	$7.5\times$ slower	

At $g_c=0$: ghost reads are free — C-stat wins

C-Stationary Col-Major: Ghost Reads Gone, Problem Remains

Fix: generate B in col-major order — all K values of column j before column $j+1$.

The FIFO output now matches C-stationary's consumption order. No ghost reads — all generated elements are used.



At $g_c=100$: still **$7.5\times$ slower** than B-stationary.

Ghost reads were not the real bottleneck.

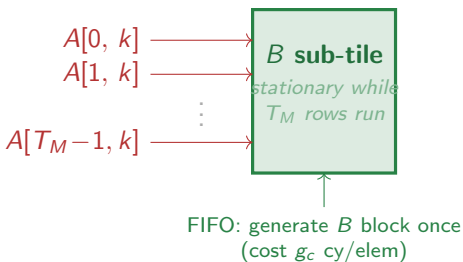
	$g_c=0$	$g_c=10$	$g_c=100$
B-stat	ref	ref	ref
C row-maj	$2\times$ fstr	$1.25\times$ fstr	$7.5\times$ slwr
C col-maj	$2\times$ fstr	$1.28\times$ fstr	$7.5\times$ slwr

The real bottleneck: C-stationary cannot amortize B generation across T_M A-rows.

B-stationary: loop order is B outer, A inner. Each B block is fetched once; all T_M A-rows sweep through it. Each B element used T_M times $\Rightarrow$ cost g_c/T_M per MAC.

C-stationary: B is consumed once per C-row pass. No T_M -fold amortization $\Rightarrow$ cost $\sim g_c$ per MAC.

B-Stationary: T_M -Fold Register Reuse



Loop order: B outer, A inner. Each B block is fetched *once*; all T_M A-rows sweep through it. Each B element is used T_M times — generation cost amortized:

$$\text{gen cost per MAC} = \frac{g_c \times N_B}{T_M \times N_B} = \frac{g_c}{T_M}$$

	$g_c=0$	$g_c=10$	$g_c=100$
B-stat	ref	ref	ref
C col-maj	$2 \times \text{fstr}$	$1.28 \times \text{fstr}$	$7.5 \times \text{slwr}$
C row-maj	$2 \times \text{fstr}$	$1.25 \times \text{fstr}$	$7.5 \times \text{slwr}$

Practical rule: use B-stationary for $g_c \gtrsim 20$; C-stationary wins only at $g_c \approx 0$ (lower α).

Naive Cycle Model: Two Cost Terms

Same reuse argument as traffic, but in **cycles**:

Source	Cost per MAC
A loaded from L1 (reused T_N times)	$\frac{C_A}{T_N}$
B generated by FIFO (reused T_M times)	$\frac{g_c}{T_M}$

Naively assuming the two costs **add**:

$$\frac{T}{MNK} = \frac{C_A}{T_N} + \frac{g_c}{T_M}$$

C_A = cycles to load one A element from L1.

g_c = cycles to generate one B element (FIFO).

$$\begin{aligned} & \text{A load: } C_A/T_N \text{ cy/MAC} \\ & + \\ & \text{B gen: } g_c/T_M \text{ cy/MAC} \\ T &= MNK \times \left(\frac{C_A}{T_N} + \frac{g_c}{T_M} \right) \\ & \text{(assumes sequential execution)} \end{aligned}$$

FIFO is Async: Two Costs in Parallel

The FIFO generates B **in the background**

while the core loads A from L1.

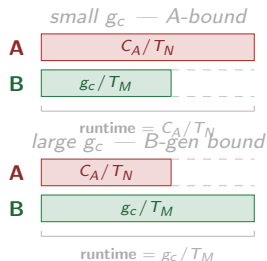
The two costs **overlap** — runtime is determined by whichever finishes last:

$$\frac{T}{MNK} = \max \left\{ \frac{C_A}{T_N}, \frac{g_c}{T_M} \right\}$$

Two regimes:

- $g_c/T_M < C_A/T_N$: A-load bound
- $g_c/T_M > C_A/T_N$: B-gen bound

Choose (T_M, T_N) to balance both terms and minimize the dominant bottleneck.



Replacing C_A/T_N with Measured $\alpha(T_M, T_N)$

The C_A/T_N term assumed:

- C_A = fixed cost per A element from L1
- T_N -fold reuse divides it out cleanly

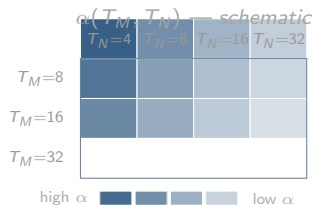
Problem: actual cycles/MAC depends on tile shape through cache residency, line utilization, and access patterns — not a simple constant divided by T_N .

Solution: replace with a measured value.

Define $\alpha(T_M, T_N) \equiv T / MNK$ at $g_c = 0$ (B generation is free):

$$\frac{T}{MNK} = \max \left\{ \alpha(T_M, T_N), \frac{g_c}{T_M} \right\}$$

α is measured once per hardware config;
the model then works for any g_c .



α decreases with T_N (more column reuse)
and depends on T_M through cache residency.

Isolating α : Set $g_c = 0$

g_c is a hardware parameter we fully control.

Key trick: set $g_c = 0$ — B generation becomes instantaneous.

The model collapses:

$$\frac{T}{MNK} = \max \left\{ \alpha(T_M, T_N), \frac{0}{T_M} \right\} = \alpha(T_M, T_N)$$

A single measurement gives α directly — no formula needed, no assumptions about C_A .

Procedure:

- 1 Run B-stationary with $g_c = 0$
- 2 Record T / MNK
- 3 That is $\alpha(T_M, T_N)$

$g_c = 0$: B generation is free

A $\alpha(T_M, T_N)$

B $\overline{\text{instantaneous}}(g_c=0)$

$$T / MNK = \alpha(T_M, T_N)$$

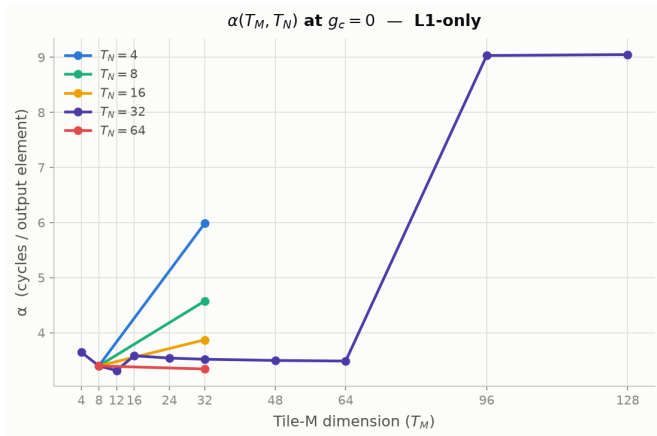
Calibration: Building the α Table

Procedure: for each valid (T_M, T_N) ,
run B-stationary with $g_c = 0$,
record T/MNK .

No formula assumed — purely empirical.

What the data shows:

- Small T_N : α spikes when the A tile overflows L1 ($T_N=4$ at $T_M=32$)
- Large T_N : α stays low until the A tile is too large for L1 ($T_N=32$ at $T_M=96$)
- α decreases with T_N (more column reuse)



Using the α Table: Predicting the Optimal Tile

$\alpha(T_M, T_N)$ is measured once and stored.

g_c/T_M is analytic — no measurement needed.

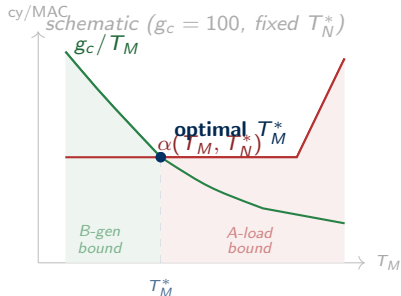
For any new g_c :

$$\text{predict}(T_M, T_N) = \max \left\{ \alpha(T_M, T_N), \frac{g_c}{T_M} \right\}$$

$$(T_M, T_N)^* = \arg \min_{T_M, T_N} \max \left\{ \alpha, \frac{g_c}{T_M} \right\}$$

No new experiments needed.

The optimal tile is where the two costs are balanced: $\alpha(T_M^*, T_N^*) \approx g_c/T_M^*$.



Roofline Validation: Setup

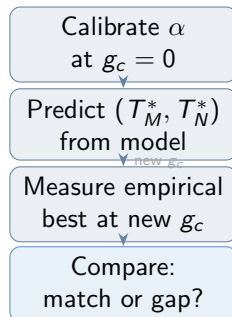
Question: calibrate α once at $g_c = 0$; does the model predict the best tile correctly at any new g_c ?

Methodology:

- 1 Measure $\alpha(T_M, T_N)$ at $g_c = 0$
- 2 For each g_c : predict $(T_M^*, T_N^*) = \arg \min \max\{\alpha, g_c / T_M\}$
- 3 Run simulator at that g_c , find empirical best
- 4 Compare prediction vs. empirical

Test conditions:

SRAM budgets	64 KB, 128 KB
Splits per budget	6 (varying L1/FIFO)
g_c values	10, 100, 250 ... 500



Repeated for all 108 conditions

Roofline Validation: Results

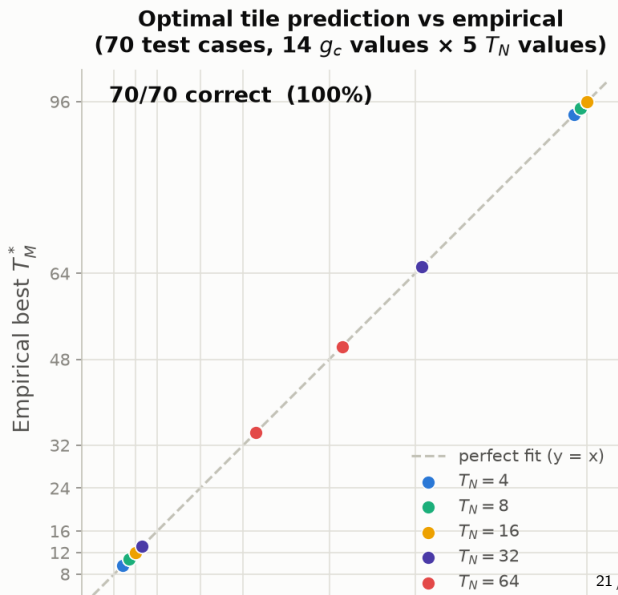
T_M^* : **always predicted correctly.**

70/70 exact matches across all g_c values and T_N choices (100%).

T_N^* : **correct up to $g_c \approx 250$,**
then degrades at high g_c .

Overall (T_M^* , T_N^*) exact match:

- $g_c \leq 250$: 100% match
- $g_c > 250$: model picks wrong T_N^*
- When wrong: cycle gap $\leq 4\%$



Why T_N^* Fails at High g_c : Gen-Bound Regime

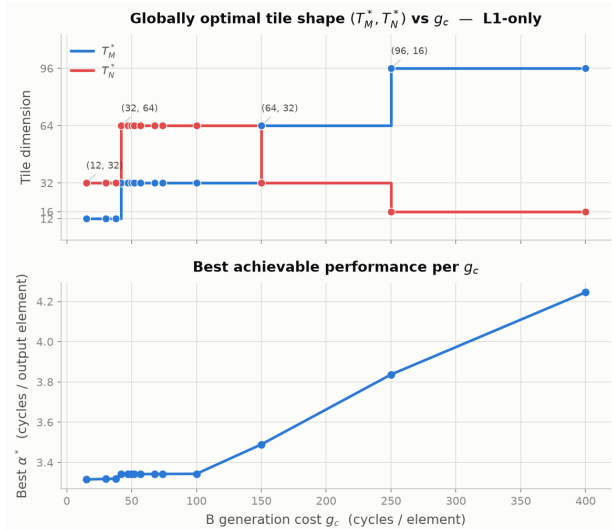
When $g_c/T_M \gg \alpha(T_M, T_N)$ for all T_N :

$$\max \left\{ \alpha, \frac{g_c}{T_M} \right\} \approx \frac{g_c}{T_M} \quad (T_N\text{-independent})$$

The model predicts the same cost for every T_N — it cannot distinguish between them.

Consequence:

- T_M^* always correct (g_c/T_M is TM-dependent)
- T_N^* chosen arbitrarily by the model
- Empirically $T_N=16$ is best in gen-bound regime (pipeline effects)



T_N^* (red) drops $64 \rightarrow 32 \rightarrow 16$ as g_c grows.

Impact of Tile Selection: Optimal vs. Square

Does choosing the right
(T_M, T_N) actually matter?

Comparing:

- Optimal asymmetric tile
(T_M^*, T_N^*)
- Square tile ($T_M = T_N$,
fixed)

Speedup from correct tile:

- $g_c = 100$: **20–40%**
faster
- $g_c = 250$: **~75%** faster
- $g_c = 400$: **up to 85%**
faster

Gain grows with g_c .

